

Continuous Control Project Report

Deep Deterministic Policy Gradient (DDPG) on Unity Reacher

1 Introduction

This report describes the solution to the Continuous Control project from the Udacity Deep Reinforcement Learning Nanodegree. The task is to train an agent to solve the Unity Reacher environment using a Deep Reinforcement Learning algorithm.

The goal of the agent is to control a double-jointed robotic arm such that the hand remains within a moving target location for as long as possible. The environment is considered solved when the agent achieves an average score of at least +30 over 100 consecutive episodes.

The final implementation successfully solved the environment in 375 episodes, reaching an average score of 30.15 over the last 100 episodes.

2 Environment description

The Unity Reacher environment consists of:

- **State space:** The state space has 33 continuous dimensions. These include positional information, rotations, velocities, and angular velocities of the robotic arm.
- **Action space:** The action space has 4 continuous dimensions. Each action corresponds to torque applied to two joints. All actions are bounded within $[-1, 1]$.
- **Reward structure:** The agent receives a reward of +0.1 for each timestep the arm's end effector remains inside the target region.
- **Solving criterion:** The environment is considered solved when the average score over 100 consecutive episodes is at least +30.

3 Learning Algorithm

3.1 Overview

The agent was trained using Deep Deterministic Policy Gradient (DDPG). DDPG is an off-policy, actor-critic algorithm designed for continuous action spaces. It combines ideas from:

- Deterministic policy gradients
- Q-learning
- Experience replay
- Target networks

The algorithm maintains:

- An actor network (policy)
- A critic network (Q-function)
- Corresponding target networks for both

Experience replay is used to break correlation between samples, and soft target updates ($\tau = 0.01$) to stabilize learning. Furthermore, states are normalized to further stabilize learning.

3.2 Network architectures

3.2.1 Actor network

- Input: 33-dimensional state
- Hidden layers:
 - Fully connected layer with 400 units (ReLU)
 - Fully connected layer with 300 units (ReLU)
 - Output layer: 4 units with tanh activation. The tanh output ensures actions remain in the valid range $[-1, 1]$.
- Weights were initialized following the original DDPG paper:
 - Hidden layers initialized uniformly in $[-1/\sqrt{f}, 1/\sqrt{f}]$
 - Final layer initialized in $[-3e-3, 3e-3]$

3.2.2 Critic Network

- Input:
 - State (33 dimensions)
 - Action (4 dimensions)
- Architecture:
 - First fully connected layer processes state only (400 units, ReLU)
 - Second fully connected layer takes concatenated state features and action (300 units, ReLU)
 - Output layer produces a single Q-value ($Q(s,a)$)

3.3 Hyperparameters

- Replay buffer size: 100,000
- Batch size: 256
- Discount factor (γ): 0.99
- Soft update parameter (τ): $1e-3$
- Actor learning rate: $2e-4$
- Critic learning rate: $5e-4$
- Hidden layers for both actor and critic network: [400, 300]

- Update frequency: 10 learning steps every 10 environment steps
- Replay warm-up: 10,000 experiences
- Gradient clipping was applied to the critic network with a maximum norm of 1.
- A maximum of 1000 timesteps per episode

3.4 Exploration strategy

The original DDPG paper proposes using an Ornstein–Uhlenbeck (OU) process for exploration noise to generate temporally correlated noise suitable for physical control problems. In this implementation, I instead used Gaussian noise:

$$\mu'(s_t) = \mu(s_t | \theta_t^\mu) + \mathcal{N}(0, \sigma^2) \times f_t$$

with $\sigma = 0.2$ scaled by a decaying noise factor f_t . The noise scaling factor starts at 1 and decays with a factor 0.999 every episode. $\mu'(s_t)$ is clipped between $[-1; 1]$

Reasoning:

- Gaussian noise is simpler to implement.
- Empirically, it performs similarly to OU noise in many modern continuous control tasks.
- The Reacher environment does not strongly require temporally correlated noise.
- Recent literature often reports negligible difference between OU and Gaussian noise in practice.

In this project, Gaussian noise produced stable learning and successful convergence. Nevertheless, it would be worthwhile to investigate the performance when OU noise would have been added.

3.5 State Normalization

A key improvement was the introduction of running state normalization. A running estimate of mean and variance across all observed states was maintained. States were normalized before being fed into the actor and critic networks:

$$s_{normalized} = \frac{s - \mu}{\sigma + \epsilon}$$

This improved training stability and learning speed by ensuring well-conditioned network inputs.

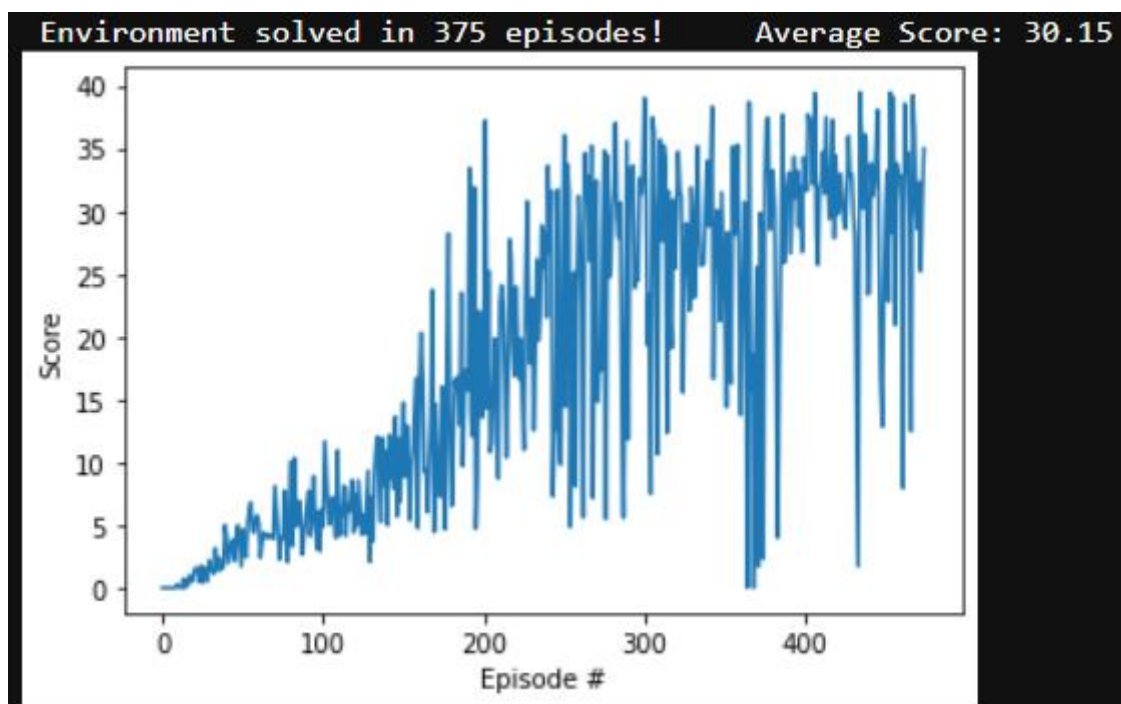
State normalization is not performed independently for each individual state sample, nor is it computed from a single minibatch. Instead, a running estimate of the mean and variance is maintained across the entire replay buffer, reflecting the global distribution of observed states. As new experiences are added, the running statistics are updated and all sampled states are normalized using the updated global mean and variance, ensuring consistency and stability throughout training.

4 Results

The agent successfully solved the environment.

- Number of episodes required: 375
- Final 100-episode average score: 30.15

The training curve shows gradual improvement followed by convergence above the +30 threshold.



5 Ideas for Future Work

Although DDPG successfully solved the task several improvements could enhance performance and reliability.

5.1 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 addresses weaknesses of DDPG:

1. Overestimation bias:
 - a. DDPG uses a single critic, which can overestimate Q-values due to maximization bias.
 - b. TD3 uses two critics and takes the minimum of the two Q-values: $Q_target = \min(Q1_target, Q2_target)$
 - c. This reduces overoptimistic value estimates.
2. Delayed policy updates:
 - a. The actor is updated less frequently than the critic.
 - b. This allows the critic to stabilize before the actor is updated.
3. Target policy smoothing
 - a. Small clipped noise is added to target actions during critic updates, reducing sharp value peaks.

TD3 typically achieves:

- Faster convergence
- Greater stability
- Lower sensitivity to hyperparameters

Replacing DDPG with TD3 would likely reduce the number of episodes needed to solve the environment.

5.2 Soft Actor-Critic (SAC)

Soft Actor-Critic is a more advanced actor-critic algorithm based on maximum entropy reinforcement learning.

Key differences from DDPG:

- Stochastic policy instead of deterministic
- Entropy term encourages exploration
- Twin Q-networks (like TD3)

The objective includes an entropy term: $J = E[\text{reward} + \alpha * \text{entropy}]$

This encourages the agent to remain exploratory while maximizing reward.

Advantages of SAC:

- Superior stability
- Better sample efficiency
- Reduced hyperparameter sensitivity
- Stronger exploration

Implementing SAC would likely:

- Solve the environment in fewer episodes
- Require less tuning
- Provide more consistent results across seeds

5.3 Additional Improvements

Other possible enhancements:

- Prioritized Experience Replay
- Layer Normalization inside networks
- Larger replay buffer
- Automatic entropy tuning (for SAC)
- Multiple random seeds for robustness evaluation
- Multi-agent version of the Reacher environment

6 Conclusion

This project demonstrated that a properly tuned DDPG implementation with:

- Experience replay
- Target networks
- Soft updates
- State normalization
- Gaussian exploration noise

can successfully solve the Unity Reacher continuous control task.

The environment was solved in 375 episodes with a final 100-episode average score of 30.15.

Future work replacing DDPG with TD3 or SAC would likely improve training speed, robustness, and stability.